Author: Madeleine Skeppås Date: 2026 06 17

GlioTrace Step-by-step User Manual

Installing GlioTrace

1. Create a new folder in your local file system and navigate to the folder inside a new terminal window. To clone the GlioTrace repository into the folder, run the following:

   ```
   git clone https://github.com/Gliomethods/GlioTrace2.0.git
   ```

2. We recommend installing GlioTrace using a virtual environment to avoid conflicts with package versions that GlioTrace depends on (e.g numpy). We use the `venv` module in Python to create a virtual environment based on Python. To create a virtual environment, run the following in the same folder you cloned the repository into:

   ```
   python3 -m venv venv
   ```

3. To activate the environment, run:

   ```
   source venv/bin/activate
   ```

4. To install the GlioTrace package, navigate to GlioTrace2.0 and run:

   ```
   pip install .
   ```

Preparing image data

GlioTrace works on numpy arrays of image data packaged into python dictionaries, a structure we call ‘stacks’. Each item in the stack is a 3d numpy array that corresponds to one of the channels in the image data, with dimensions **height x width x time.**

Use the function `select_stabilize.py` to convert your image data into stacks. This function allows optional region selection (when you want to select smaller regions to analyse) and stabilization (to correct for imaged specimen drift between time-points).

In the Example folder, open `process_image_example.py` in an editor. Define the following parameters:

- Input file path – path to TIFs
  - Note: select_stabilize expects file paths organized in one of two ways:
    - File path is one folder with TIFs from same experiment
    - File path contains subfolders with TIFs from separate experiments
  - Note about folder names: each individual experiment folder should be named ‘exp_{number}’ to match the metadata (example further down).
- Output path – output location for image stacks
- Json path – output location for your stack list specifying the path to each stack
- Mode:
  - Manual – manual region selection from an interactive window
  - Coords – region selection using a file containing coordinates (see example below)

- Full – no region selection needed

The script in the Example folder contains one example for each **mode**, but you only have to run one of them. When you have specified the parameters, save the script and run the following in the console:

```
python process_image_example.py
```

Generated stacks and a stackfile listing the newly created stacks will end up in the output path.

Running GlioTrace

0. Create a metadata file
GlioTrace requires some minimal metadata info about each experiment number to be given as input. See example below on how this file should be structured.

1. Create a GlioTrace object
To run GlioTrace on your stacks, open in the Example folder the notebook `class_run_example.ipynb` and specify the path to your stackfile (created in the previous step) and metadata file. Optionally, you can change which features you'd like to test in the logistic model, by changing the `fcols` parameter. After these are specified, you can create a GlioTrace object:

```
gliobj = GlioTrace(stackfile=stackfile,
                           metadata=metadata,
                           fcols=fcols)
```

For more examples on how to filter on patient ID, treatment arms etc., see notebook.

2. Run cell tracking
To run cell tracking, call the object method `run_tracking`. It takes parameters for saving and re-loading tracking objects, suitable if you want to resume a crashed run. You can also specify detection sensitivity to adjust cell detection sensitivity, as well as a detection-backup parameter for a second sensitivity level if the first one generates empty cell detections.

```
gliobj.run_tracking(save_point="save_dir/run1", load_point="savedir/run1")
```

3. Fit HMM

To estimate the parameters of the HMM model, such as the transition matrix and feature effects, as well as the quantified uncertainty around them via jackknife, run the object method `loo_hmm` once the tracking is complete. Here, you can re-enter which features you would like to test. Add optional treatment-interaction terms to be tested, if you expect the effect of a feature to differ between treated and control cells. Additionally, you can specify a grouping parameter for the leave-one-out model fits. If not given, this defaults to set as a grouping if such a column exists in the metadata, otherwise experiment_id. If you'd just like to fit the HMM for one cell line, perturbation or dose (or a specific combination), this can be specified in the parameters. If nothing is specified, the method

will run separately for each unique combination of perturbation, dose and cell line, from what can be found in the metadata.

```
results = gliobj.loo_hmm(
    fcols=["vascular_distance", "polarization", "tme_label", "is_treatment"],
    save_path="runs/loo/"
)
```

For each unique combination, `loo_hmm` automatically saves one full model and one model per LOO replicate to the specified save_path. It also saves the results to the same directory, which is strcutured like a dict keyed by the unique combination, with transition matrices and jackknife uncertainty estimates per feature. Each dict item contains:

- global_a – the global transition matrix from the full data
- global_a_mean – the mean of the global transition matrices across the jackknife runs
- global_a_var – the variance of the global transition matrices across the jackknife runs
- trans_diff – the difference in transition probabilities between 90th and 10th percentile of the treatment feature (e.g. "is_treatment") for each feature in fcols + interaction terms (full data)
- trans_diff_mean – the mean of the trans_diff across the jackknife runs
- trans_diff_var – the variance of the trans_diff across the jackknife runs

Metadata example

Each original image stack (e.g. a folder of 3d TIF-files) needs to have an experiment ID, taken from the folder name during the image preparation step. This experiment ID is used to map the experiment to some minimal metadata required by GlioTrace, which is the imaging interval in hours, called ‘delta_t’ in the metadata file. Should you then select smaller regions, will they automatically be associated with the original experiment ID, and get an additional region of interest (ROI) ID. You don’t have to specify a metadata row for each ROI, only one per experiment number.
If you want to filter your experiments on patient ID or treatment/ control status, you can also add that information here and GlioTrace will handle that in the creation of the object (see examples in the notebook).

Metadata file example 1 (contains only the minimal information required)

experiment_id	delta_t
1	1,17
2	2

Metadata file example 2 (contains extra information on treatment, dose and patient origin of the biological material used in the experiment, as well as a technical grouping parameter called set)

experiment_id	set	delta_t	patient_id	perturbation	dose	unit
1	1	1,17	patient 1	control	0	uM
2	1	2	patient 1	drug 1	5	uM

Author: Madeleine Skeppås Date: 2026 06 17

ROI coordinate table example

If you already have a set of pre-defined coordinates for regions of interest in your imaged sample, GlioTrace can automatically extract and stabilize these regions if provided with a coordinate table defined as below:

exp	X	Y	W	H
1	756	385	500	500
1	258	958	500	500
2	127	374	500	500
2	647	835	500	500